Quintero Rubio Martin

PRÁCTICA 4

MANEJO DE UNA TABLA DE VERDAD DENTRO DE UN PROCESS.

OBJETIVO:

El alumno aprenderá el manejo de una tabla de verdad dentro de la instrucción process y verificará que la lista de sensibilidad está dada por las entradas en todo sistema combinacional.

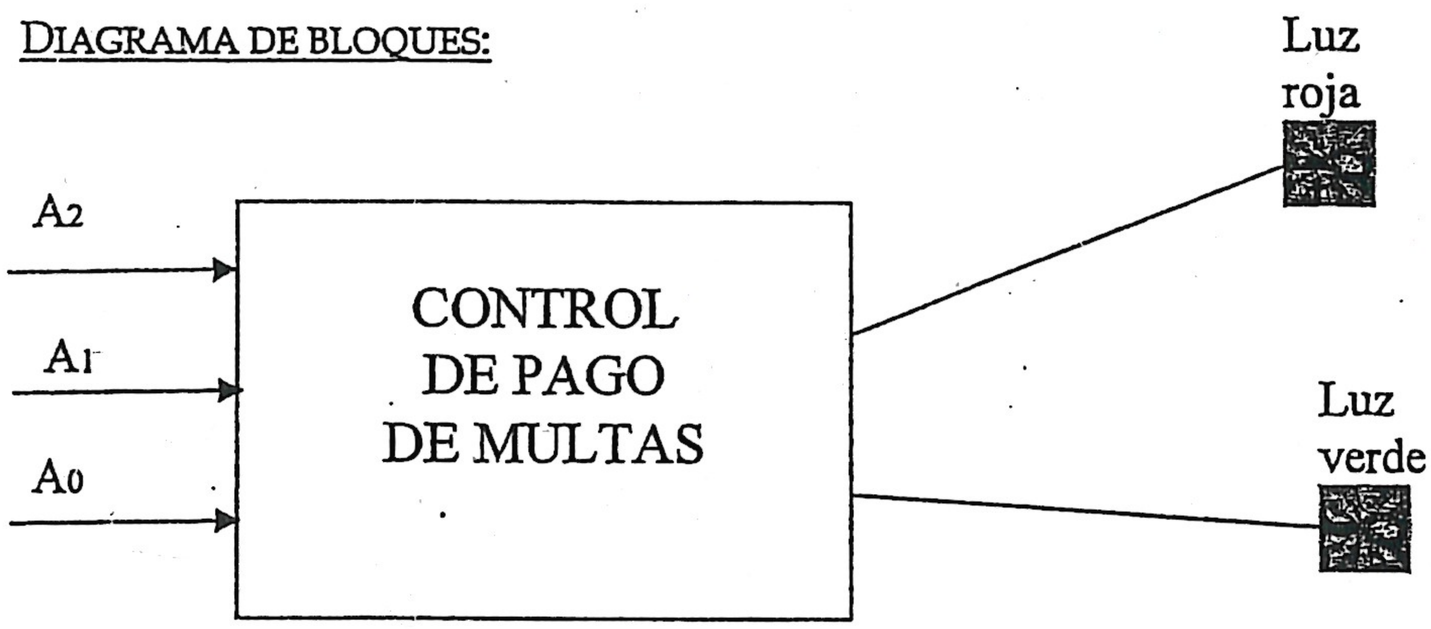
ACTIVIDADES:

Diseñar el control de pago de multas que maneje dos luces (roja y verde) utilizando la instrucción process en lenguaje VHDL.

ESPECIFICACIONES:

Si la luz roja se prende el infractor paga la multa y si la luz verde se prende no la paga, las luces no se pueden prender simultáneamente. Este sistema tiene por entrada la decisión de un jurado compuesto por tres personas (A_2,A_1,A_0) cuyas opiniones valen: A_2=35%, A_1=30%, A_0=15%. Un uno lógico a la entrada, significa que ese juez desea que el culpable pague la multa.
La multa se paga cuando el jurado quiere que sea pagada por un 50% o más. La figura 4.1 muestra el diagrama de bloques.

DIAGRAMA DE BLOQUES:



4.1 Diagrama de Bloques del sistema de pago de multas

A_0	A_1	A_2
		0
		1
		0
		1

35+30 = 65
35+15+30 = 8
35+

A_0 11

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;
entity must is
    Port ( A : in  STD_LOGIC_VECTOR (1
downto 0);

           sal : out  STD_LOGIC_VECTOR (6
downto 0));
end must;

architecture Behavioral of must is
begin

process ( A)
begin
  case A is
      when "00" => sal <= "0001001";
      when "01" => sal <= "1000000";
      when "10" => sal <= "1000111";
      when "11" => sal <= "0001000";
    when others => sal <= "0000000";
   end case;
end process;
end Behavioral;
```

Figura 8.2 Código en lenguaje VHDL de un BUSMUX.

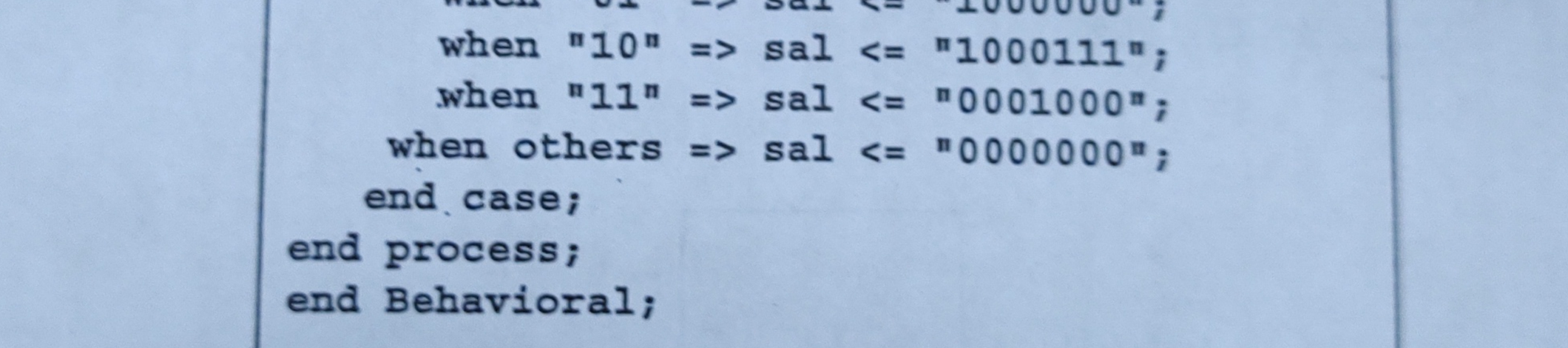


Figura 8.3 Simulación del BUSMUX.

PRÁCTICA 8

DISEÑO DE UN BUS MUX

OBJETIVO:

El alumno aprenderá el manejo del flujo de datos en un multiplexor.

ESPECIFICACIONES:

Diseñar y construir un selector cuya salida sea un vector de 7 bits de forma tal que con las cuatro combinaciones distintas que se tienen en el vector de selección de entrada A(A1,A0), en un display de siete segmentos se vea la letra H con las líneas de selección en 00, la letra O con las líneas de selección en 01, la letra L con las líneas de selección en 10 y la letra A con las líneas de selección en 11. Se utiliza lógica negada para la asignación binaria a los vectores de salida.
En la figura 8.1 se observa el diagrama de bloques requerido, en la figura 8.2 el código VHDL y su simulación se muestra en la figura 8.3

DIAGRAMA DE BLOQUES:

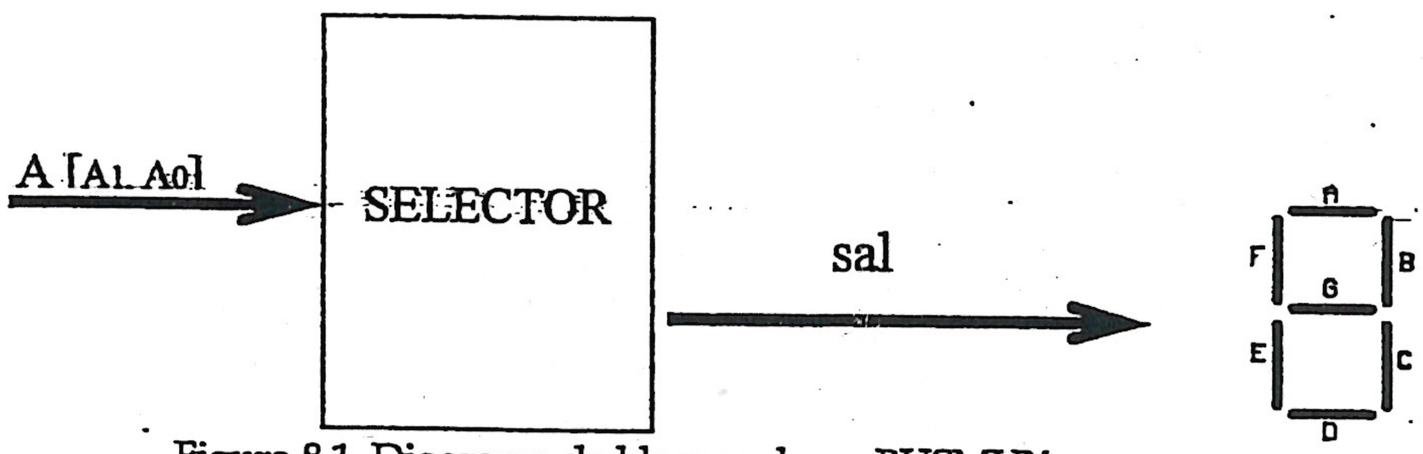


Figura 8.1. Diagrama de bloques de un BUSMUX.

El código en VHDL de este sistema se observa en la figura 4.2, tomando la entrada como un vector A de tres bits (A_2, A_1, A_0) y la figura 4.3 muestra su simulación.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

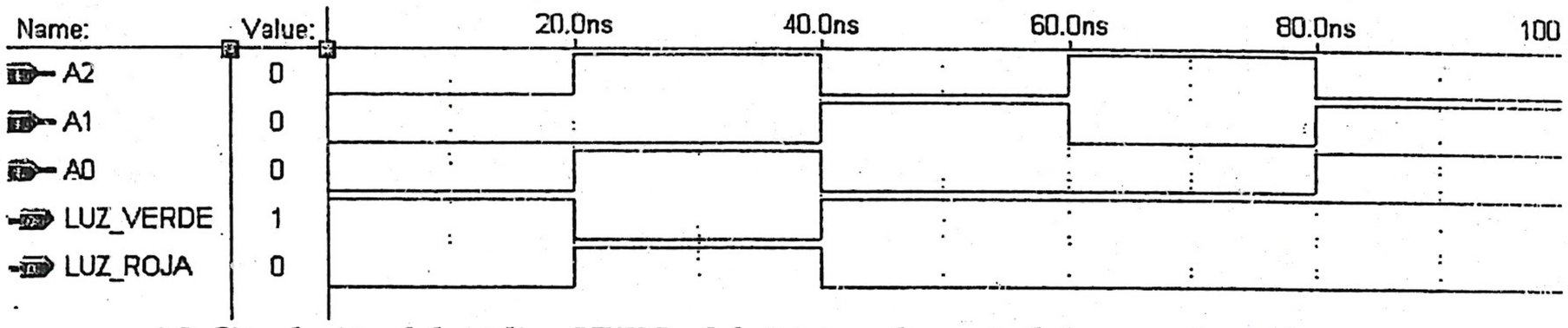
entity PAGO is
   Port ( A : in  STD_LOGIC_VECTOR (2 downto 0);
          LUZ_ROJA : out  STD_LOGIC;
          LUZ_VERDE : out  STD_LOGIC);
end PAGO;

architecture Behavioral of PAGO is
begin
process(A)
begin
case A is
                when "000" =>
                Luz_Roja <= '0'; Luz_Verde <= '1';
                when "001"=>
                Luz_Roja <= '0'; Luz_Verde <= '1';
                when "010" =>
                Luz_Roja <= '0'; Luz_Verde <= '1';
                when "011" =>
                Luz_Roja <= '0'; Luz_Verde <= '1';
                when "100" =>
                Luz_Roja <= '0'; Luz_Verde <= '1';
                when "101" =>
                Luz_Roja <= '1'; Luz_Verde <= '0';
                when "110" =>
                Luz_Roja <= '1'; Luz_Verde <= '0';
             when others =>
             Luz_Roja <= '1'; Luz_Verde <= '0';
end case;

end process;

end Behavioral;
```

4.2. Código VHDL del sistema de control de pago de multas.



4.3. Simulación del código VHDL del sistema de control de pago de multas.